FREELANCE MARKETPLACE

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

submitted by

GEETHAM VENKATA SIVA SAI P

23501A0544

Under the guidance of

Mr. A. Prashant, Asst. Prof.

Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled **"Freelance Marketplace"** that is being submitted by **Geetham Venkata Siva Sai P (23501A0544)**, as part of Assignment-1 and Assignment-2 for the **Blockchain Technology** course during the academic year **2026-27**. The report is prepared on the topic of decentralized freelance work coordination and escrow payments using the Ethereum blockchain and Solidity smart contracts.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
ASSIGNMENT-2: ____/5

INDEX

S. No.	Chapter	Page No.
1	Introduction	01
2	Background	03
3	Blockchain and Ethereum Basics	04
4	Use Case Overview	06
5	System Architecture and Workflow	09
6	Implementation Using Solidity and Ethereum	12
7	Benefits	17
8	Challenges and Security Considerations	18
9	Conclusion and Future Enhancements	20
10	SDGs Addressed	21
11	References	22
12	Appendix A	23

1. INTRODUCTION

1.1. Overview of Blockchain Technology

Blockchain is a distributed ledger maintained by a network of computers. Transactions are grouped into blocks and linked cryptographically, creating a history that is difficult to alter without detection. Public blockchain networks allow participants to verify transactions without relying exclusively on a single central database. Ethereum extends this model by supporting programmable smart contracts that can hold state and execute rules when transactions are submitted. [1]

Beyond recording transfers of value, blockchains with smart-contract support can coordinate agreements between parties who do not fully trust one another. Because the rules of a contract are visible and execute identically on every node, participants can inspect the logic before committing funds. Ethereum, the platform used in this report, is one of the most widely used networks for this purpose. [1]

1.2. Relevance of Blockchain to Freelance Marketplaces

Traditional freelance marketplaces typically depend on a centralized platform to manage job listings, client–freelancer communication, payment processing, and dispute procedures. Users may face delayed payouts, unclear fee structures, account restrictions, or limited visibility into how a payment decision was made. A blockchain-enabled marketplace can record important job and payment events on Ethereum and use a Solidity smart contract to hold funds until agreed completion conditions are met.

Freelance work carries risk for both sides: a client who pays in advance may not receive the work, and a freelancer who works first may not be paid. An escrow arrangement reduces this mutual risk by holding the payment in a neutral place until the agreed condition is satisfied. Implementing that neutral place as a smart contract makes the rules explicit and the payment path auditable by both parties.

1.3. Proposed Solution

The proposed Freelance Marketplace is a web application that connects clients who publish jobs with freelancers who submit proposals and complete work. For a scoped prototype, a client creates a job and deposits the agreed amount of Ether into an escrow smart contract. A freelancer is assigned to the job, and the client can release payment after accepting the deliverable. The contract emits events so the application can display a verifiable transaction history. The blockchain component is intended to improve payment transparency; it does not automatically guarantee work quality or resolve every dispute.

1.4. Objectives

The objectives of this use case are to:

- provide transparent job and payment records;
- demonstrate escrow using Solidity on Ethereum;

- reduce reliance on manual payment coordination;
- define role-based actions for clients and freelancers; and
- build a practical foundation that can later support proposals, milestones, ratings, and dispute handling.

1.5. Organization of the Report

Chapter 2 describes the challenges of existing freelance marketplaces and the reasons for choosing Ethereum and Solidity. Chapter 3 summarizes the blockchain and Ethereum concepts needed to follow the design. Chapter 4 defines the use case, scope, actors, and requirements. Chapter 5 presents the system architecture and workflow, and Chapter 6 describes the implementation approach, including an illustrative smart contract and an honest status check of the project repository. Chapters 7 and 8 discuss the benefits, challenges, and security considerations. Chapter 9 concludes the report and outlines future enhancements, Chapter 10 maps the work to the United Nations Sustainable Development Goals, and Chapters 11 and 12 contain the references and the appendix.

2. BACKGROUND

Online freelance marketplaces have made it easier to find remote work, but the way they handle trust and payment raises recurring concerns. This chapter reviews those concerns, explains the escrow concept, and outlines why Ethereum and Solidity were selected for the proposed design.

2.1. Existing Freelance Marketplace Challenges

Centralized platforms provide useful discovery and communication features, but users may encounter platform fees, delayed withdrawals, account-level dependency, limited portability of reputation, and disputes about whether work meets the agreed scope. Payment protection often depends on the platform's internal policies and the availability of support staff.

Because job history, reputation, and payment records are held inside a single platform, a freelancer who leaves or is removed from that platform may lose the record of past work. Clients, in turn, may be unsure how their funds are protected if the freelancer does not deliver.

2.2. Payment Trust and Escrow

Escrow is a mechanism in which payment is held until a defined condition is met. In a smart-contract design, funds can remain at a contract address and be released only by an authorized transaction that satisfies the contract rules. This can make the payment path auditable. The design must still consider cancellation, non-delivery, partial completion, and disagreements; a simple escrow contract should not be represented as a complete dispute-resolution service.

2.3. Roles in the Proposed System

Clients publish work and fund jobs. Freelancers browse available work, accept an assignment or are selected by a client, and submit deliverables through the application. The smart contract records the job status and holds the escrowed amount. The web application provides search, forms, status views, and wallet connectivity. A backend may store larger descriptions and file references, while the blockchain stores only the minimum information needed for payment logic and auditability.

2.4. Why Ethereum and Solidity

Ethereum is a programmable blockchain with a mature ecosystem of wallets, development tools, test networks, and smart-contract standards. [1] Solidity is a statically typed language designed for Ethereum Virtual Machine (EVM) contracts. [2] Ethereum-compatible development tools such as Hardhat, Remix, MetaMask, and ethers.js can be used to compile, test, deploy, and interact with the contract. [4] [5] [6] This report uses Ethereum and Solidity concepts; it does not use Aptos or the Move language.

2.5. Need for a Blockchain-Based Approach

Taken together, these concerns create a need for a payment step that is transparent, rule-based, and independent of any single operator’s internal records. A smart-contract escrow does not replace a marketplace’s search, communication, or support functions; instead, it complements them by making the payment step verifiable by both parties.

3. BLOCKCHAIN AND ETHEREUM BASICS

Ethereum is a public, programmable blockchain on which applications are implemented as smart contracts. Understanding a small set of core concepts is enough to follow the design of the proposed marketplace. Below are the key concepts that define the platform:

3.1. Decentralized Ledger

A blockchain replicates transaction data across participating nodes. Each node verifies transactions according to the network's rules. A public Ethereum transaction can be inspected using a block explorer, making contract interactions independently auditable.

3.2. Smart Contracts

A smart contract is code deployed to the blockchain at an address. Its functions execute when transactions call them, and its state changes are recorded on-chain. Contract execution is deterministic: the same transaction and state produce the same result under the network rules. A contract cannot independently browse websites or determine whether a submitted file is good quality; such judgments require trusted input or a separately designed process. [2]

3.3. Solidity Language Concepts

Solidity includes contract definitions, state variables, structs, mappings, arrays, functions, modifiers, events, custom errors, and visibility controls. A marketplace contract can represent each job using a struct containing the client, freelancer, payment amount, status, and timestamps. Mappings can associate job identifiers with job records. Modifiers or explicit checks restrict sensitive actions to the relevant client or assigned freelancer. [2]

3.4. Ether, Gas, and Transactions

Ether (ETH) is Ethereum's native asset. Users pay gas for transaction execution, and the amount depends on computation, storage operations, and network conditions. A transaction that creates a job or releases escrow requires a wallet signature and may incur a fee. Read-only calls can generally be queried without sending an on-chain transaction, although infrastructure providers may impose their own limits. [1]

3.5. Wallets and Addresses

A wallet such as MetaMask manages a user's private keys and signs transactions. [6] The account address identifies the caller to the smart contract, but it is not proof of a person's real-world identity. Users must protect seed phrases and verify contract addresses before approving transactions. The application should explain when a transaction is about to move funds.

3.6. Ethereum Development Lifecycle

A typical lifecycle includes writing the Solidity contract, compiling it, running unit and adversarial tests, deploying to a local chain, testing on an Ethereum test network such as Sepolia, reviewing the code, and only then considering a production deployment. [7] Deployment is not equivalent to a security audit, and a test-network demonstration should be clearly labelled as such.

3.7. Consensus and Finality

Ethereum uses a proof-of-stake consensus mechanism in which validators that have staked ETH propose and attest to blocks; the network moved from proof of work to proof of stake in 2022. A transaction is usually included in a block within seconds, but it is considered finalized only after further confirmations, which on Ethereum mainnet takes roughly thirteen minutes. For a marketplace this distinction matters: the interface should present a payment as complete only after the transaction is confirmed, which is why this report repeatedly distinguishes pending, confirmed, and failed states. [1]

By combining a replicated ledger, deterministic smart contracts, wallet-based signatures, and a widely supported toolchain, Ethereum provides the building blocks needed for a rule-based escrow. These building blocks do not remove the need for careful contract design, testing, and review, which are discussed in later chapters.

4. USE CASE OVERVIEW

The use case for the Freelance Marketplace is to connect clients and freelancers while making the payment lifecycle easier to verify. This chapter states the objectives and scope of the system, identifies the actors, and lists the functional and non-functional requirements.

4.1. Use Case Objectives

The system aims to connect clients and freelancers while making the payment lifecycle easier to verify. A job is created with a description, assigned freelancer, and ETH amount. The contract records that the job is funded. After the client confirms completion, the payment is released to the freelancer and the status becomes completed. The contract should reject unauthorized or invalid state transitions.

4.2. Scope of the System

Included in the prototype:

- job creation and funding;
- assignment of a freelancer;
- viewing job status;
- client-authorized payment release;
- contract events; and
- wallet-based transaction signing.

The interface may also provide job browsing, proposal forms, profile details, and file links off-chain.

Excluded from the minimal escrow contract:

- automatic quality assessment;
- legal arbitration;
- guaranteed identity verification;
- fiat conversion; and
- fully decentralized file storage.

4.3. Main Actors

Actor	Role in the system
Client	Posts work, deposits funds, and confirms completion.
Freelancer	Views jobs, performs work, and submits a delivery notification through the application.
Smart contract	Enforces payment and status rules.
Frontend	Presents the user experience and sends transactions.
Ethereum network	Validates and records contract calls.
Optional backend / database	Stores long descriptions, search indexes, user profile data, and file metadata without treating that database as the source of truth for escrow balance.

4.4. Functional Requirements

ID	Requirement
FR-1	A client can create a job with a non-zero payment.
FR-2	The job is associated with the client and intended freelancer.
FR-3	The contract holds the payment until an authorized release.
FR-4	Only the client can release payment for the client’s job.
FR-5	A job cannot be completed or paid twice.
FR-6	The system emits events for job creation, assignment, delivery, and payment release.
FR-7	The interface displays transaction status and a link to the relevant block explorer.
FR-8	Invalid requests display a useful error without suggesting that an unsuccessful transaction succeeded.

4.5. Non-Functional Requirements

Quality	Requirement
Security	Use access checks, reentrancy-aware payment patterns, and input validation. [3]
Reliability	Test success and failure paths.
Usability	Explain wallet prompts and gas fees.
Transparency	Show job identifiers, state, and transaction hashes.
Maintainability	Separate frontend logic from contract code.
Privacy	Avoid putting personal information, private messages, or confidential work files directly on a public blockchain.

4.6. Illustrative Usage Scenario

The following scenario summarizes how a client (Client A) and a freelancer (Freelancer B) would use the proposed system:

1. Client A connects a wallet, enters the job details, names Freelancer B, and funds the job with the agreed amount of ETH.
2. The contract stores the job with the status Funded and emits a JobCreated event.
3. Freelancer B performs the work and delivers it through the application; the file itself stays off-chain.
4. Client A reviews the delivery and, if satisfied, submits a release transaction.
5. The contract checks the caller and status, updates the state, transfers the ETH to Freelancer B, and emits a PaymentReleased event.
6. Both parties verify the result using the transaction hash on a block explorer.

In the minimal design, if the client rejects the work or never responds, the contract provides no automatic recovery path. Cancellation, time-limited review, and dispute handling are therefore identified as future work.

5. SYSTEM ARCHITECTURE AND WORKFLOW

This chapter presents the layered architecture of the proposed marketplace, the lifecycle of a job from creation to payment, and the division of data between the blockchain and conventional storage.

5.1. Architecture Layers

The proposed system is organized into the following layers:

- **User layer:** clients and freelancers use the web interface.
- **Application layer:** React or another web framework renders job listings, profiles, and job details.
- **Wallet layer:** MetaMask signs Ethereum transactions.
- **Contract layer:** Solidity escrow logic validates state transitions and holds ETH.
- **Ethereum layer:** the EVM executes contract calls and records state changes.
- **Optional data layer:** a conventional database stores searchable profile information and large text or file references.

Figure 5.1 shows how these layers connect: the browser-based interface requests signatures from MetaMask, an Ethereum RPC provider submits the signed transaction to the escrow contract, and a block explorer or event indexer helps inspect receipts.

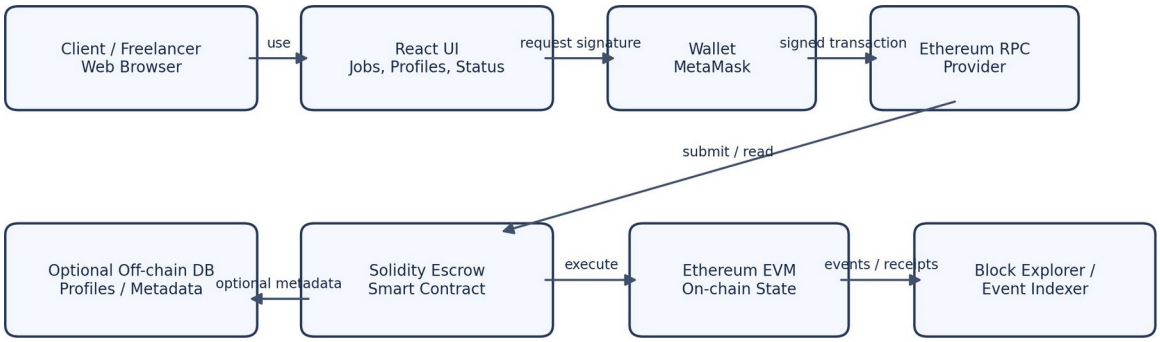


Figure 5.1: Proposed Ethereum-based Freelance Marketplace architecture

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

5.2. Job Lifecycle

The client connects a wallet, enters job details, selects or later assigns a freelancer, and funds the job. The contract creates a unique job record and emits an event. The freelancer completes the agreed work and marks it delivered through the application if that action is enabled in the contract. The client reviews the delivery and submits a release transaction. If the transaction is accepted, the contract transfers the escrowed amount to the freelancer and marks the job complete. If a dispute process is not implemented, users should not assume the contract can recover funds automatically when the parties disagree. The complete flow is illustrated in Figure 5.3 (Section 5.4).

5.3. Data Placement

On-chain data should be limited to what is required for contract execution and verification, such as wallet addresses, payment amount, status, and timestamps. Detailed descriptions, personal profiles, messages, and work files are better kept off-chain with appropriate access controls. A content hash or file identifier can be recorded on-chain when integrity verification is needed, but a hash does not make the underlying file private. Figure 5.2 summarizes this split between on-chain, off-chain, and frontend responsibilities.

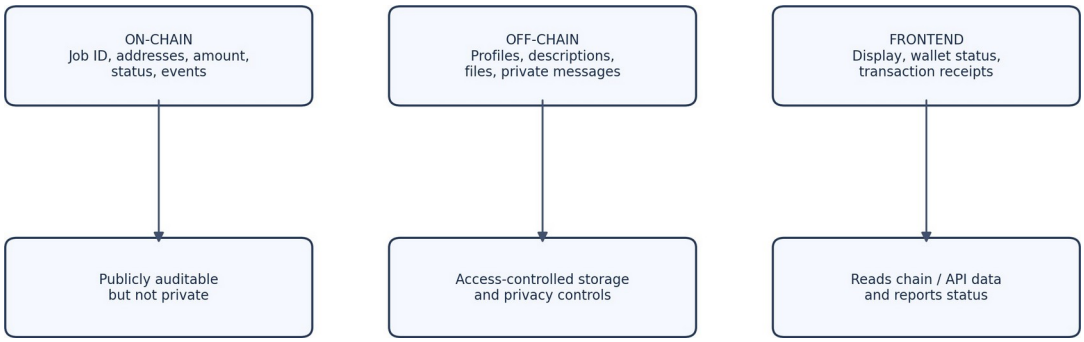


Figure 5.2: On-chain vs off-chain data placement

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

5.4. Workflow Diagram

Figure 5.3 shows the proposed job-to-payment workflow, from wallet connection and escrow funding through submission, client approval, ETH transfer, and the final status update in the interface. The interface should report success only after the transaction is confirmed.

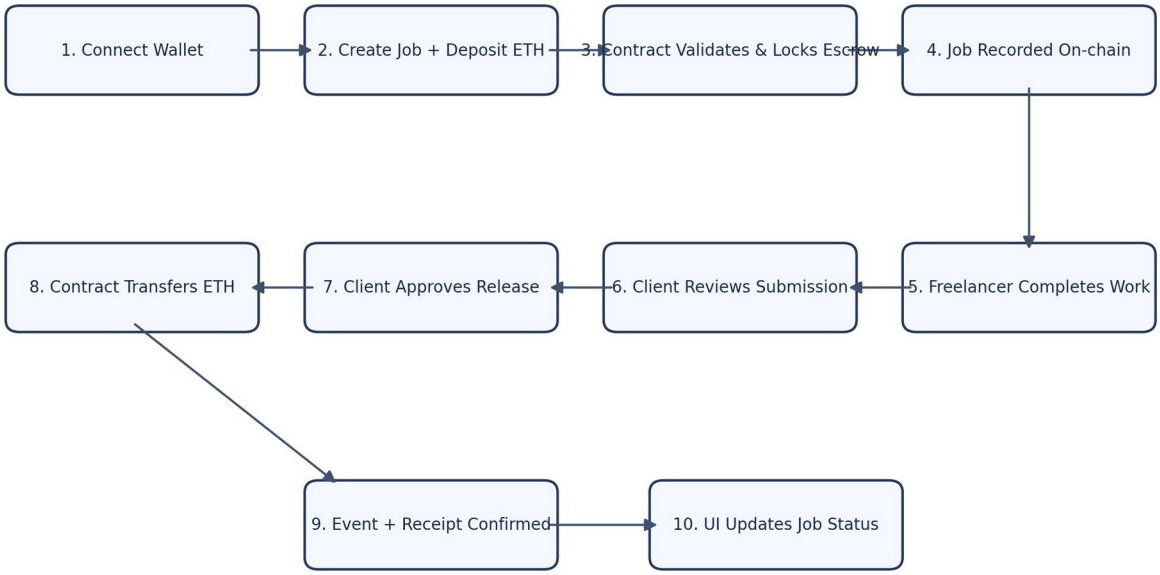


Figure 5.3: Proposed job-to-payment workflow

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

5.5. Technology Stack

Table 5.1 summarizes the technologies proposed for each part of the system.

Component	Technology	Purpose
Frontend	React or another web framework	Job listings, forms, and status views
Wallet	MetaMask [6]	Account management and transaction signing
Client library	ethers.js [5]	Contract calls from the browser
Smart contract	Solidity 0.8.x [2]	Escrow logic and job state
Development tools	Remix, Hardhat [4]	Compile, test, and deploy
Networks	Local chain, Sepolia test network [7]	Testing and demonstration
Optional off-chain storage	Database and file storage	Descriptions, profiles, and file references

Table 5.1: Proposed technology stack

6. IMPLEMENTATION USING SOLIDITY AND ETHEREUM

This chapter describes how the proposed escrow design can be implemented with Solidity and Ethereum tooling. The contract listing and diagrams describe a proposed implementation; Section 6.10 compares them with the evidence found in the project repository.

6.1. Development Environment

A suitable implementation environment is Solidity 0.8.x with Remix or Hardhat, an Ethereum local development network for unit testing, MetaMask for wallet interaction, and ethers.js for frontend contract calls. [2] [4] [5] [6] Sepolia can be used for test deployment. [7] Tool versions should be pinned in the project and recorded in the final implementation log. Goerli is deprecated and should not be selected for a new test deployment.

6.2. Job Data Structure

A Job struct can store the client address, freelancer address, amount in wei, status, and created timestamp. A mapping from job ID to Job provides direct lookup. An enum such as Open, Assigned, Delivered, Completed, and Cancelled can make state transitions explicit. Every function should validate the current state before modifying it.

6.3. Creating and Funding a Job

A payable createJob function receives ETH with the transaction. It checks that the payment is non-zero and the assigned freelancer is a valid address, allocates an identifier, stores the job, and emits a JobCreated event. The amount stored should equal msg.value, so the contract does not rely on a user-supplied amount that differs from the actual funds received. Figure 6.1 shows the intended message sequence for job creation; note that transaction submission and transaction confirmation are separate events, and the interface must wait for confirmation.

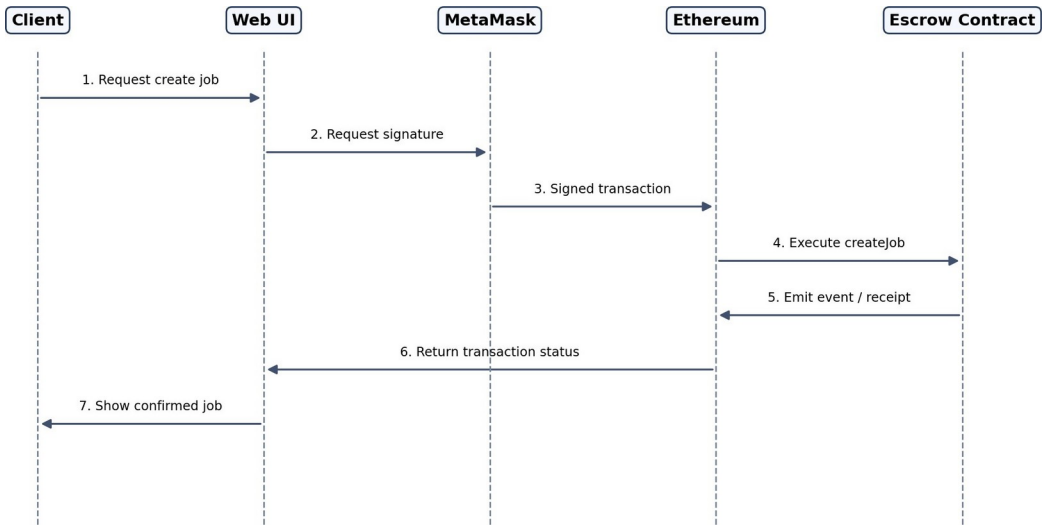


Figure 6.1: Transaction sequence for job creation (proposed)

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

6.4. Releasing Escrow

A releasePayment function checks that the caller is the client, the job is in a deliverable state, and the escrow amount has not already been released. It updates the state before making the external transfer and uses a safe transfer pattern, such as a low-level call with a checked return value. For production use, this logic should be reviewed and tested against reentrancy and unexpected recipient behaviour. [3] Figure 6.2 shows the resulting state transitions: a job moves from Funded to Completed only when the authorized client releases payment, while invalid callers, invalid states, and failed ETH transfers revert.

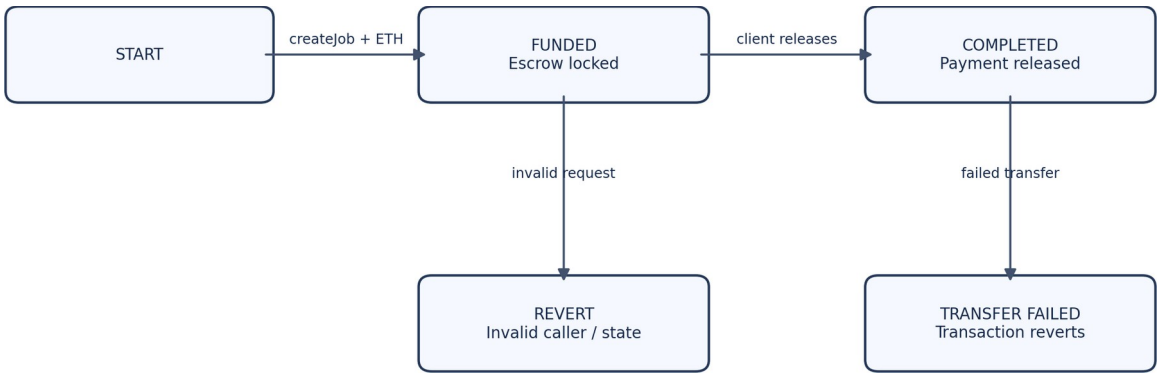


Figure 6.2: Escrow contract state transitions

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

6.5. Events and Application Integration

Events such as JobCreated, WorkDelivered, and PaymentReleased let a frontend or indexing service track changes. The application should wait for transaction confirmation and show the resulting transaction hash. A button click is not proof that the transaction succeeded; wallet rejection, insufficient funds, RPC failure, or contract reversion must be handled explicitly. Figure 6.3 outlines the validation and failure-handling path, in which the interface distinguishes pending, confirmed, and failed states rather than showing success immediately.

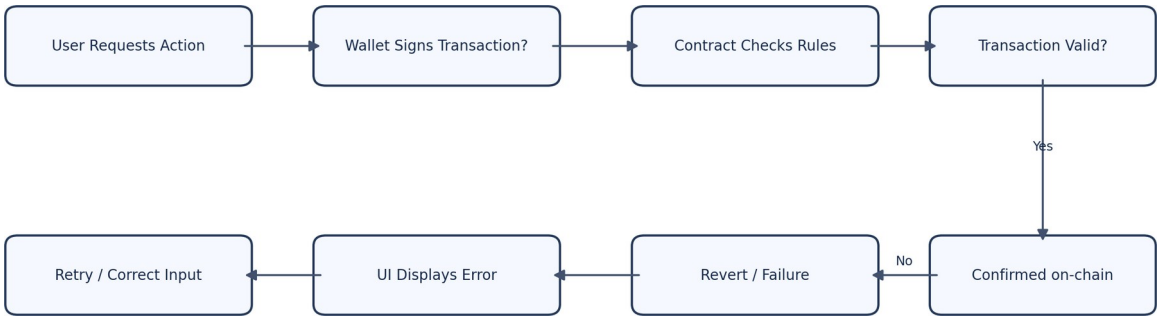


Figure 6.3: Transaction validation and failure handling

Proposed Ethereum/Solidity design; not evidence of a deployed or tested system.

6.6. Example Solidity Contract

The following listing illustrates a minimal escrow pattern for academic demonstration. It has not been compiled, deployed, audited, or tested as part of the inspected repository. It omits cancellation, arbitration, deadlines, and milestone payments; do not use it

to hold real funds without independent review. For brevity, the listing uses only the Funded and Completed states from the fuller status set described in Section 6.2.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract FreelanceMarketplace {
    enum Status { Funded, Completed }

    struct Job {
        address payable client;
        address payable freelancer;
        uint256 amount;
        Status status;
    }

    uint256 public nextJobId;
    mapping(uint256 => Job) public jobs;

    event JobCreated(uint256 indexed jobId, address indexed client,
        address indexed freelancer, uint256 amount);
    event PaymentReleased(uint256 indexed jobId, address freelancer,
        uint256 amount);

    function createJob(address payable freelancer)
        external payable returns (uint256 jobId) {
        require(freelancer != address(0), "Invalid freelancer");
        require(freelancer != msg.sender, "Client cannot be freelancer");
        require(msg.value > 0, "Escrow must be non-zero");

        jobId = nextJobId++;
        jobs[jobId] = Job({
            client: payable(msg.sender),
            freelancer: freelancer,
            amount: msg.value,
            status: Status.Funded
        });

        emit JobCreated(jobId, msg.sender, freelancer, msg.value);
    }

    function releasePayment(uint256 jobId) external {
        Job storage job = jobs[jobId];
        require(job.client == msg.sender, "Only client");
        require(job.status == Status.Funded, "Not funded");

        uint256 amount = job.amount;
        job.status = Status.Completed;
        job.amount = 0;
```

```
        (bool ok, ) = job.freelancer.call{value: amount}("");
        require(ok, "Transfer failed");

        emit PaymentReleased(jobId, job.freelancer, amount);
    }
}
```

Code interpretation and validation

- createJob is payable and stores msg.value as the escrow amount, checks the freelancer address, creates a job record, and emits JobCreated.
- releasePayment checks that the caller is the client and that the job is still funded. It updates state before transferring Ether and checks the transfer result.
- Required tests: successful funding, zero-value rejection, unauthorized release, double-release prevention, invalid addresses, and freelancer receipt. Compile and test with a pinned Solidity version before use.

6.7. Example Tests

Unit tests should cover successful job creation, rejection of zero-value jobs, rejection of unauthorized payment release, prevention of double release, and payment receipt by the freelancer. Additional tests should cover zero addresses, unexpected state transitions, failed recipient transfers, and any cancellation or dispute logic added later. Tests on a local chain are necessary but do not replace an independent security review.

6.8. Frontend Integration

The frontend connects to MetaMask, checks the active chain ID, creates a contract instance using the deployed address and ABI, and calls the relevant functions. It should not ask users to paste private keys into the website. The deployed contract address and network must be displayed clearly, and the interface should provide a way to inspect the transaction on an Ethereum block explorer.

6.9. Deployment Procedure

Compile and test locally first. Deploy to Sepolia using a funded test wallet, verify the deployed address and constructor parameters, and exercise all user flows. Record the compiler version, deployment transaction, chain ID, contract address, and test results. Do not deploy an unreviewed escrow contract holding real funds. Any production version should receive independent review and clear operational procedures for emergencies and upgrades, if upgrades are supported.

6.10. Implementation Status Verification

This audit compares the documented target design with the inspected project repository. A feature described in the report is not, by itself, evidence that the feature is implemented. Table 6.1 summarizes the findings.

Feature	Status	Evidence / action needed
Marketplace concept / UI	Partially present	Repository describes marketplace UI and job form; end-to-end behavior still needs verification.
Ethereum / Solidity escrow	Not verified	Repository search did not locate a Solidity contract. Implement, compile, and test it.
Wallet integration	Aptos-oriented	Inspected form uses Aptos wallet imports; Ethereum target needs MetaMask/provider integration.
On-chain payments	Mock behavior observed	Inspected aptosClient.ts returns simulated transaction hashes and a fixed sample balance.
Architecture and workflow	Documented / proposed	The diagrams in this report illustrate the intended Ethereum design, not a verified deployment.
Automated tests and deployment	Not provided / not verified	Add test output, chain ID, contract address, and transaction receipts only after testing.
Dispute resolution / quality assessment	Not included in minimal design	Treat as future work unless separately implemented and demonstrated.

Table 6.1: Implementation status of the project repository

Repository links

- Project: https://github.com/Geetu2050/Hackathon-1
- README: https://github.com/Geetu2050/Hackathon-1/blob/main/README.md
- Solidity code search: https://github.com/Geetu2050/Hackathon-1/search?q=solidity&type=code

Before publication

- Compile the contract and record the compiler version.
- Test valid funding, invalid value/address, unauthorized release, double release, and recipient payment.
- Deploy to Sepolia with test ETH and record the chain ID, contract address, transaction hashes, and receipts.

- Record actual gas use and test outcomes; do not invent performance results.
- Obtain an independent security review before any production use.

Publication integrity note: the Solidity listing and diagrams describe a proposed implementation. The inspected repository evidence does not establish that Ethereum/Solidity escrow is operational.

7. BENEFITS

A smart-contract escrow offers several advantages over payment arrangements that depend entirely on a platform's private records. Below are the key advantages of the proposed design, with the limits of each noted where relevant:

7.1. Transparent Payment History

On-chain transactions provide a public audit trail of escrow funding and release. Clients and freelancers can inspect the transaction history instead of relying solely on a private platform record.

7.2. Rule-Based Escrow

The contract can enforce that payment is not released by an arbitrary third party and that only the authorized client can trigger the agreed release flow. Explicit state checks help prevent duplicate completion and double payment.

7.3. Reduced Dependence on Manual Payment Coordination

Once deployed, the payment logic can execute according to the contract's rules without a platform operator manually initiating each transfer. This does not remove the need for support, dispute handling, or off-chain communication.

7.4. Global Accessibility

A wallet and internet connection can allow users in different locations to interact with the same contract. However, users remain subject to local laws, network availability, and the practical costs of using Ethereum.

7.5. Verifiable Events

Contract events can be indexed by the application to show when a job was created or payment released. The transaction hash allows users to independently verify that the corresponding action was included on-chain.

7.6. Extensibility

The architecture can later add milestone-based payments, proposals, reputation records, arbitration integrations, stablecoin support, or a Layer 2 network. Each extension introduces additional design and security requirements and should be implemented incrementally.

In summary, the proposed design improves the visibility and predictability of the payment step. Its benefits are strongest where the agreed conditions are simple and objectively checkable, and they should be weighed against the challenges described in the next chapter.

8. CHALLENGES AND SECURITY CONSIDERATIONS

Smart-contract escrow also introduces technical, security, and legal challenges that must be addressed before the system is relied on. Below are the key challenges:

8.1. Gas Costs and Network Congestion

Ethereum transactions require gas and fees vary with network demand. Frequent small actions may be expensive for a marketplace. A Layer 2 network can reduce costs, but it changes the deployment and bridging assumptions and still requires users to understand the selected network.

8.2. Smart Contract Vulnerabilities

Common risks include reentrancy, missing access controls, incorrect state transitions, unsafe external calls, arithmetic or accounting errors, and denial-of-service patterns. Solidity 0.8.x includes checked arithmetic by default, but this does not prevent logical bugs. Use established patterns, automated tests, static analysis, and an independent audit before handling valuable funds. [3]

8.3. Disputes and Subjective Work Quality

A contract cannot determine whether a design, report, or software feature satisfies a subjective brief without trusted evidence and decision rules. The minimal design relies on client approval. A more complete system may add milestones, a time-bound review window, a mutually selected arbitrator, or an appeal process; these mechanisms need careful governance and should not be claimed as implemented unless they exist.

8.4. Privacy and Confidentiality

Public chain data is visible to observers. Do not place real names, addresses, private messages, confidential deliverables, or sensitive client details directly on-chain. Use access-controlled off-chain storage and record only minimal references or hashes when needed.

8.5. Key Management and User Errors

Lost or compromised wallet keys can prevent legitimate access or allow unauthorized transactions. The application should explain wallet security, show the destination and amount before confirmation, and never request seed phrases. A smart contract generally cannot distinguish the rightful person from another party controlling the same private key.

8.6. Legal, Tax, and Operational Constraints

Freelance work may involve employment classification, consumer protection, taxation, sanctions, intellectual-property rights, and contract enforceability. Blockchain automation does not replace legal advice or a clear service agreement. The project should describe the prototype’s limits and seek appropriate review before real-world deployment.

8.7. Limitations of the Prototype

The proposed minimal contract demonstrates job creation, escrow funding, and client-authorized payment release. It does not by itself provide decentralized identity, automated quality assessment, guaranteed dispute resolution, fiat payment support, or secure file storage. Those features require separate implementation and evaluation.

8.8. Summary of Risks and Mitigations

Table 8.1 pairs each major risk with the mitigation proposed in this report.

Risk	Proposed mitigation
Gas costs	Keep on-chain actions minimal, show fee estimates before signing, and evaluate a Layer 2 network.
Contract vulnerabilities	Use access checks, state-before-transfer ordering, established libraries [3], thorough tests, static analysis, and an independent audit.
Disputes and subjective quality	Consider milestones, review windows, or arbitration only after their governance is designed.
Privacy	Keep personal data and files off-chain; record only hashes or references when needed.
Key management	Educate users, display destination and amount before confirmation, and never request seed phrases.
Legal and operational	Use a clear service agreement and obtain appropriate review before real-world use.
Prototype limits	State the scope clearly and treat missing features as future work.

Table 8.1: Risks and proposed mitigations

Despite these challenges, a carefully scoped and tested escrow contract can still improve payment transparency. To realize that benefit safely, the project must prioritize security review, honest labelling of what is and is not implemented, and clear user guidance.

9. CONCLUSION AND FUTURE ENHANCEMENTS

9.1. Conclusion

This report presents a Freelance Marketplace concept using Ethereum and Solidity to demonstrate transparent job records and rule-based escrow payments. The client funds a job through a payable smart-contract function, and the contract releases payment only when an authorized completion action is performed. Events and transaction hashes allow the application to display verifiable activity. The approach can improve auditability, but its usefulness depends on careful contract design, usability, dispute procedures, and security testing.

9.2. Future Enhancements

Future work may introduce milestone escrow, client and freelancer proposals, ratings with abuse controls, a time-limited acceptance period, optional arbitration, and more complete cancellation rules. The interface could support WalletConnect and Ethereum Layer 2 networks. A reputation system should account for fake reviews and sybil accounts. Off-chain storage should use access controls, backups, and clear retention policies. Before production, the contract should undergo independent audit and a threat-model review.

9.3. Suggested Development Roadmap

A staged approach is recommended so that each feature is verified before the next is added:

1. Implement, compile, and test the minimal escrow contract on a local development chain.
2. Connect the frontend to MetaMask and replace simulated transaction behavior with real contract calls.
3. Deploy to the Sepolia test network and record the chain ID, contract address, and transaction hashes.
4. Add cancellation, time-limited review, and milestone payments, with tests for each new state transition.
5. Commission an independent security review before any use with real funds.

By following this path and continuing to label proposed features separately from verified ones, the Freelance Marketplace can evolve from an academic prototype into a more dependable and widely usable solution.

10. SDGs ADDRESSED

The Freelance Marketplace relates to several United Nations Sustainable Development Goals (SDGs). The contributions below are potential rather than guaranteed, and each depends on how the system is implemented and used.

SDG 8: Decent Work and Economic Growth

A transparent freelance marketplace can support access to independent work opportunities and provide a clearer record of payment events. This is a potential contribution rather than a guarantee of fair work or economic outcomes.

SDG 9: Industry, Innovation and Infrastructure

Using smart contracts and web-based tools demonstrates digital infrastructure for coordinating work and payments. A secure implementation requires reliable software engineering, testing, and ongoing maintenance.

SDG 10: Reduced Inequalities

A globally accessible interface may lower some geographic barriers to finding clients. Access still depends on connectivity, device availability, language, wallet usability, and local legal conditions.

SDG 12: Responsible Consumption and Production

Digital workflow and records can reduce some paper-based administration. The environmental impact of a blockchain application depends on the network and transaction architecture; the report does not claim that every blockchain design is automatically sustainable.

11. REFERENCES

[1] Ethereum Foundation. Ethereum Documentation. Visited in October 2026, from https://ethereum.org/en/developers/docs/

[2] Solidity Team. Solidity Documentation. Visited in October 2026, from https://docs.soliditylang.org/

[3] OpenZeppelin. Contracts: Security and Access Control. Visited in October 2026, from https://docs.openzeppelin.com/contracts/

[4] Hardhat. Hardhat Documentation. Visited in October 2026, from https://hardhat.org/docs

[5] ethers.js. Documentation. Visited in October 2026, from https://docs.ethers.org/

[6] MetaMask. Developer Documentation. Visited in October 2026, from https://docs.metamask.io/

[7] Ethereum.org. Sepolia Testnet. Visited in October 2026, from https://ethereum.org/en/developers/docs/networks/

12. APPENDIX A

A.1. Suggested Project Structure

```
freelance-marketplace/
    contracts/
        FreelanceMarketplace.sol
    test/
        FreelanceMarketplace.test.js
    scripts/
        deploy.js
    frontend/
        src/
            components/
            pages/
            services/
    hardhat.config.js
    package.json
```

A.2. Suggested Demonstration Checklist

1. Connect a test wallet to the selected Ethereum test network.
2. Create a job and fund it with test ETH.
3. Confirm the job ID and funding transaction.
4. Attempt an unauthorized release and confirm it reverts.
5. Release payment from the authorized client account.
6. Confirm the freelancer receives the expected amount.
7. Verify the completed state and event logs.
8. Record the contract address, chain ID, transaction hashes, and test results.

A.3. Important Note

This is an academic prototype report and example contract, not an audited financial product. Test with testnet funds only until the contract has been reviewed, tested, and approved for the intended deployment environment.

A.4. Project Files (Google Drive)

https://drive.google.com/drive/folders/1rp-FfCtDqW7wbj5GZXCRCcCr780wVVJr?usp=sharing

A.5. Github

https://github.com/Geetu2050/BCT